Can a Tiny AI on an 8 GB MacBook Write Real Business Logic?

Locating the spec-dialect boundary where a 3-billion-parameter coder model stops producing correct code

Muhammad Junaid — iteration, ladder experiments & repair loops · **Muhammad Ansar** — original research & problem · 2 July 2026

ABSTRACT

We ask whether `Qwen2.5-Coder-3B-Instruct-4bit`, run locally on Apple MLX, can generate a `process_calculations()` that scores a perfect 12/12 on a ten-rule pricing-and-commission specification. The model is fixed; the only lever is the *spec*. A re-dialected spec (v2) closes the gap from the original's 1/12 to **12/12 on the first greedy attempt**. We then build an eight-rung *spec ladder* — same ten rules and same targets, varying only the phrasing — to find the exact point of failure. The break is sharp and surprising: it is not the two constructs originally blamed (a multi-tier conditional and a truthy-string boolean), but the removal of the literal lookup tables, after which the 3B can no longer assemble the multi-step *commission* calculation. Three control rungs confirm it: keep the tables and the model tolerates every "trap." Finally, a white-box repair loop that feeds per-step intermediate diffs rescues the boolean rung to 12/12 at a low temperature, but does not reliably rescue the fully de-scaffolded rung — reaffirming that for a small model the dependable lever is the specification, not a cleverer repair loop.

1 The question and the setup

Small, locally-hosted language models are attractive: no data leaves the laptop, no per-token bill, and an 8 GB Apple Silicon MacBook can host a 3B model comfortably. The open question is whether such a model can produce *numerically correct* business logic — not plausible-looking code, but code whose output matches ground truth to the cent, on every row.

The task is fixed throughout. A model must emit a Python function `process_calculations(csv_path)` that applies a ten-rule pricing and commission ruleset to each row of a CSV and returns the running total `total_receivables`. Success is **12/12**: all twelve ground-truth rows matched within a 0.5 tolerance. The model is fixed at `Qwen2.5-Coder-3B-Instruct-4bit` (the *Coder* variant, via `mlx-lm`); we never swap in a larger or different model. Every experiment holds the model, the harness, and the arithmetic constant, and changes only the wording of the spec.

2 The initial win — and why the old solution failed

A five-rule version of the spec, each rule a single unambiguous operation, is solved by the 3B on the first try: **12/12, no repairs.** The trouble begins at ten rules. Running the article's original *True/False* spec, the model's best attempts are **3/12 on the 3B** and **1/12 on the 7B** — the architecture is right but every row is 2–5% off. Two failures, both *linguistic* rather than a capability ceiling, explain it:

- **Rule 4 — the volume tier.** The spec's three-way tier (`<100 → 0%`, `<1000 → 5%`, `≥1000 → 10%`) is collapsed by the model into `0.05 if v<100 else (0.1 if v<1000 else 0.1)` — wrong in two of three branches.

- **Rule 10 — the boolean.** The `tax verified` cell is the string `"False"`, which is *truthy* in Python, so `not row["tax verified"]` skips withholding at exactly the rows where it should apply.

The fix (the "v2" spec) re-expresses the *same ten rules* in the model's dialect: Rule 4 becomes two independent additive checks (`if v>=100: +0.05`; `if v>=1000: +0.05`) with an explicit "no elif"; the boolean becomes `yes` / `no` text in both spec and data; rates are handed over as literal Python dictionaries and bare decimals. The arithmetic is bit-identical to the original (maximum absolute difference in targets = 0.0). The result: **12/12 on the first greedy attempt**. Same problem, phrased so the model cannot trip — the lever is the spec, not the model.

3 The sweet-spot experiment

The v2 spec passes and the original fails, but those are two distant points. *Where exactly* is the boundary, and how technical must a spec author be to stay on the right side of it? To answer precisely we build a ladder of eight specs. Every rung describes the identical ten-rule problem against identical targets; the harness is frozen at the configuration that produced the validated 12/12 (same instruction wrapper, same decoding, same scorer). **Only the spec file changes**, so any change in outcome is attributable to the spec's dialect alone.

Rungs L1–L5 strip one layer of authoring scaffolding at a time, from the heavily-engineered v2 dialect down to the natural original. Rungs L6–L8 are *controls*: they keep the literal lookup tables but re-arm the volume-tier and boolean "traps," to test whether those traps are what actually break the model.

Greedy first: the deterministic result

We score each spec two ways. The simplest is to run it once with **greedy** decoding (temperature 0), which always takes the single most-likely next token — so the *same prompt yields the identical program every time*, giving one reproducible score per spec. The prescriptive spec (**L1**) passes a perfect **12/12** this way, reproducing the v2 deliverable's headline; every less-scaffolded rung fails a single greedy shot.

Rung	Spec dialect	Greedy score	Result
L1	Prescriptive pseudocode	12/12	PASS
L2	Declarative spec	0/12	fail
L3	Business prose	0/12	fail
L4	Natural rules (tier)	0/12	fail
L5	Original dialect (boolean)	4/12	fail
L6	Tables + natural tier	0/12	fail
L7	Tables + True/False	0/12	fail
L8	Tables + tier + True/False	0/12	fail

Table 1 · Greedy (deterministic) — one temperature-0 attempt per spec. The prescriptive anchor L1 passes; the rest fall short in a single deterministic shot (L5 reaches 4/12, the others 0).

Why one shot isn't enough

A greedy shot is a knife-edge: it carries whatever bug the single most-likely completion happens to have, and a trivial rewording can flip it — a one-line change to a spec's *title* once flipped L1 from 12/12 to 0/12. To read a spec's real quality we therefore **sample**: at temperature 0.4 the same prompt produces *different* code on each run, so we ask each spec ten times and record the **best of the ten** (can it reach 12/12 at all? — "reachable") and the **pass-rate** (how many of the ten were fully correct). This sampled view — not the brittle single greedy shot — is what the rest of the paper uses.

Figure 1 (next page) shows all eight specs as the model sees them, scaled to a single page. The prose after it walks through each rung.

Figure 1 · The eight-rung spec ladder (exactly the text fed to the model, HTML authoring notes stripped)

L1 · Prescriptive pseudocode

```
# Pricing & commission calculation spec
Build `process_calculations(csv_path)`. It reads a CSV and returns a list of dicts, one
per row, each containing the key `total_receivables`.
## Reading the CSV
Normalize every row before applying the rules — strip surrounding spaces from the headers
and **lower-case every text value** so comparisons work regardless of case. Copy this exactly:
```
row = {key.strip(): value.strip().lower() for key, value in row.items()}
```
This turns e.g. `EMEA` into `emea` and `non-EMEA` into `non-emea`. Then convert `volume`
and `productid` to numbers, and treat an empty cell as its stated default.
## Input columns
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : "yes" or "no" (an empty cell counts as "no")
## Lookup tables (copy these exactly)
```
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}
```
## The 10 rules (apply in this exact order)
1. **Rate.** Look up `rate = PRODUCT_RATES[productid][status]`.
2. **Base.** `receivable_before_discount = volume * rate`.
3. **Spring discount rate.** Start `spring_rate = 0.0`. If `season` is "spring", set `spring_rate = 0.05`.
4. **Volume discount rate.** Build it up with two independent checks (do NOT use elif):
   Start `volume_rate = 0.0`.
   If `volume >= 100`, add `0.05` to `volume_rate`.
   If `volume >= 1000`, add another `0.05` to `volume_rate`.
   (So volume under 100 gives 0.0, 100–999 gives 0.05, 1000 or more gives 0.10.)
5. **Strategic discount rate.** Start `strategic_rate = 0.0`. If `customer` is "strategic", set `strategic_rate = 0.05`.
6. **Apply discounts.**
   `total_discount_rate = spring_rate + volume_rate + strategic_rate`.
   `receivable_after_discount = receivable_before_discount * (1 - total_discount_rate)`.
7. **Region loading.** Start `region_loading = 0.0`. If `region` is "emea", set `region_loading = 0.10 * receivable_after_discount`.
8. **Payment loading.** Start `payment_loading = 0.0`. If `payment type` is "credit-card", set `payment_loading = 0.025 * receivable_after_discount`.
9. **Commission.**
   Look up `multiplier = CATEGORY_MULTIPLIER[product category]`.
   `commission = (region_loading + payment_loading) * multiplier`.
10. **Tax withholding and total.**
    `subtotal = receivable_after_discount + commission`.
    Start `withholding = 0.0`.
    If `tax verified` is "no" (or the cell was empty), set `withholding = 0.03 * subtotal`.
    If `tax verified` is "yes", leave `withholding = 0.0`.
    `total_receivables = subtotal - withholding`.
Output: a list of dicts, one per input row, each with the key `total_receivables`.
```

L2 · Declarative spec

```
# Pricing & commission calculation spec
`process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the
key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : "yes" or "no" (an empty cell counts as "no")
## Lookup tables
```
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}
```
## The calculation
The base receivable is `volume * PRODUCT_RATES[productid][status]`.
Three discount rates then apply, and they add together:
- a spring rate of `0.05` when `season` is "spring", otherwise `0.0`;
- a volume rate that is `0.05` for `volume >= 100` plus an additional `0.05` for
  `volume >= 1000` (the two thresholds are independent and stack, so `volume >= 1000`
  earns the full `0.10`);
- a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`.
The receivable after discount is the base times `(1 - sum of those three rates)`.
Two loadings are computed from the receivable after discount:
- a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`;
- a payment loading of `0.025 * receivable_after_discount` when `payment type` is
  "credit-card", else `0.0`.
The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`.
The subtotal is `receivable_after_discount + commission`. Tax withholding is
`0.03 * subtotal` when `tax verified` is "no" (or empty), and `0.0` when it is "yes".
`total_receivables = subtotal - withholding`.
```

L3 · Business prose

```
# Pricing & commission calculation spec
Write `process_calculations(csv_path)` that reads the CSV and returns one dict per row
with the key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : "yes" or "no" (an empty cell counts as "no")
## The calculation
The per-unit price depends on `productid` and `status`. For productid 1 it is $1,000 when
status is individual and $900 when corporate; for productid 2 it is $1,500 / $1,200; for
productid 3 it is $2,000 / $1,700. The base receivable is `volume` times that per-unit price.
Then three discounts come off, and they stack (add together):
- **5%** if the `season` is spring;
- a volume discount of **5%** once `volume` reaches 100, plus a further **5%** once
  `volume` reaches 1000 (so 100–999 is 5% and 1000-or-more is 10%);
- **5%** if the `customer` is strategic.
Apply the combined discount to get the receivable after discount.
Two surcharges are then figured from the receivable after discount:
- **10%** if the `region` is EMEA;
- **2.5%** if the `payment type` is credit-card.
Commission is the sum of those two surcharge amounts, scaled by a factor that depends on
`product category`: electronics ×1.2, groceries ×0.8, luxury ×1.5.
The subtotal is the receivable after discount plus the commission. If `tax verified` is
"no" (or blank), withhold **3%** of the subtotal; if it is "yes", withhold nothing.
`total_receivables` is the subtotal minus the withholding.
```

L4 · Natural rules (tier)

```
# Pricing & commission calculation spec
Write `process_calculations(csv_path)` that reads the CSV and returns one dict per row
with the key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : True or False (an empty cell counts as False)
## The calculation
The per-unit price depends on `productid` and `status`. For productid 1 it is $1,000 when
status is individual and $900 when corporate; for productid 2 it is $1,500 / $1,200; for
productid 3 it is $2,000 / $1,700. The base receivable is `volume` times that per-unit price.
Then three discounts come off, and they stack (add together):
- **5%** if the `season` is spring;
- a volume discount based on how much was ordered: under 100 units gets no volume
  discount, 100 to 999 units gets 5%, and 1000 units or more gets 10%;
- **5%** if the `customer` is strategic.
Apply the combined discount to get the receivable after discount.
Two surcharges are then figured from the receivable after discount:
- **10%** if the `region` is EMEA;
- **2.5%** if the `payment type` is credit-card.
Commission is the sum of those two surcharge amounts, scaled by a factor that depends on
`product category`: electronics ×1.2, groceries ×0.8, luxury ×1.5.
The subtotal is the receivable after discount plus the commission. The `tax verified`
column is True or False: when the sale has not been tax-verified, withhold 3% of the
subtotal; when it has been verified, withhold nothing.
`total_receivables` is the subtotal minus the withholding.
```

L5 · Original dialect (boolean)

```
# Pricing & commission calculation spec
Write `process_calculations(csv_path)` that reads the CSV and returns one dict per row
with the key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : True or False (an empty cell counts as False)
## The calculation
The per-unit price depends on `productid` and `status`. For productid 1 it is $1,000 when
status is individual and $900 when corporate; for productid 2 it is $1,500 / $1,200; for
productid 3 it is $2,000 / $1,700. The base receivable is `volume` times that per-unit price.
Then three discounts come off, and they stack (add together):
- **5%** if the `season` is spring;
- a volume discount based on how much was ordered: under 100 units gets no volume
  discount, 100 to 999 units gets 5%, and 1000 units or more gets 10%;
- **5%** if the `customer` is strategic.
Apply the combined discount to get the receivable after discount.
Two surcharges are then figured from the receivable after discount:
- **10%** if the `region` is EMEA;
- **2.5%** if the `payment type` is credit-card.
Commission is the sum of those two surcharge amounts, scaled by a factor that depends on
`product category`: electronics ×1.2, groceries ×0.8, luxury ×1.5.
The subtotal is the receivable after discount plus the commission. The `tax verified`
column is True or False: when the sale has not been tax-verified, withhold 3% of the
subtotal; when it has been verified, withhold nothing.
`total_receivables` is the subtotal minus the withholding.
```

L6 · Tables + natural tier

```
# Pricing & commission calculation spec
`process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the
key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : "yes" or "no" (an empty cell counts as "no")
## Lookup tables
```
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}
```
## The calculation
The base receivable is `volume * PRODUCT_RATES[productid][status]`.
Three discount rates then apply, and they add together:
- a spring rate of `0.05` when `season` is "spring", otherwise `0.0`;
- a volume rate based on how much was ordered: `0.0` under 100 units, `0.05` from 100 to
  999 units, and `0.10` at 1000 units or more;
- a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`.
The receivable after discount is the base times `(1 - sum of those three rates)`.
Two loadings are computed from the receivable after discount:
- a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`;
- a payment loading of `0.025 * receivable_after_discount` when `payment type` is
  "credit-card", else `0.0`.
The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`.
The subtotal is `receivable_after_discount + commission`. Tax withholding is
`0.03 * subtotal` when `tax verified` is "no" (or empty), and `0.0` when it is "yes".
`total_receivables = subtotal - withholding`.
```

L7 · Tables + True/False

````
# Pricing & commission calculation spec
`process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the
key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : True or False (an empty cell counts as False)
## Lookup tables
```
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}
```
## The calculation
The base receivable is `volume * PRODUCT_RATES[productid][status]`.
Three discount rates then apply, and they add together:
- a spring rate of `0.05` when `season` is "spring", otherwise `0.0`;
- a volume rate that is `0.05` for `volume >= 100` plus an additional `0.05` for
  `volume >= 1000` (the two thresholds are independent and stack, so `volume >= 1000`
  earns the full `0.10`);
- a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`.
The receivable after discount is the base times `(1 - sum of those three rates)`.
Two loadings are computed from the receivable after discount:
- a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`;
- a payment loading of `0.025 * receivable_after_discount` when `payment type` is
  "credit-card", else `0.0`.
The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`.
The subtotal is `receivable_after_discount + commission`. The `tax verified` column is
True or False: withhold `0.03 * subtotal` when the sale has not been tax-verified, and
`0.0` when it has been verified.
`total_receivables = subtotal - withholding`.
````

L8 · Tables + tier + True/False

````
# Pricing & commission calculation spec
`process_calculations(csv_path)` reads a CSV and returns one dict per row, each with the
key `total_receivables`.
## Input columns
Each row has these columns (header names may have extra surrounding spaces (strip leading/trailing whitespace only,
and keep inner spaces like in `payment type`); compare text in lower
case; convert `volume` and `productid` to numbers; an empty cell means the stated default):
- `productid` : "1", "2", or "3"
- `status` : "individual" or "corporate"
- `volume` : an integer
- `season` : "spring", "summer", "winter", or "autumn"
- `customer` : "strategic" or "non-strategic"
- `region` : "emea" or "non-emea"
- `payment type` : "ach/wire" or "credit-card"
- `transaction type` : "domestic", "international", or "cross-border"
- `product category` : "electronics", "groceries", or "luxury"
- `tax verified` : True or False (an empty cell counts as False)
## Lookup tables
```
PRODUCT_RATES = {
    1: {"individual": 1000, "corporate": 900},
    2: {"individual": 1500, "corporate": 1200},
    3: {"individual": 2000, "corporate": 1700},
}
CATEGORY_MULTIPLIER = {"electronics": 1.2, "groceries": 0.8, "luxury": 1.5}
```
## The calculation
The base receivable is `volume * PRODUCT_RATES[productid][status]`.
Three discount rates then apply, and they add together:
- a spring rate of `0.05` when `season` is "spring", otherwise `0.0`;
- a volume rate based on how much was ordered: `0.0` under 100 units, `0.05` from 100 to
  999 units, and `0.10` at 1000 units or more;
- a strategic rate of `0.05` when `customer` is "strategic", otherwise `0.0`.
The receivable after discount is the base times `(1 - sum of those three rates)`.
Two loadings are computed from the receivable after discount:
- a region loading of `0.10 * receivable_after_discount` when `region` is "emea", else `0.0`;
- a payment loading of `0.025 * receivable_after_discount` when `payment type` is
  "credit-card", else `0.0`.
The commission is `(region_loading + payment_loading) * CATEGORY_MULTIPLIER[product category]`.
The subtotal is `receivable_after_discount + commission`. The `tax verified` column is
True or False: withhold `0.03 * subtotal` when the sale has not been tax-verified, and
`0.0` when it has been verified.
`total_receivables = subtotal - withholding`.
````

Walking the ladder

- **L1 — Prescriptive pseudocode** (mirrors v2). Literal dicts, additive Rule 4 with "no elif", `yes/no` boolean, bare decimals, step-by-step recipe. The anchor: **reachable, 12/12.**
- **L2 — Declarative spec.** Drops the imperative "`Start x = 0.0` … do NOT use elif" recipe; states *what* each rule computes in plain prose. Tables still handed over. **Reachable, 12/12** — the recipe was not load-bearing.
- **L3 — Business prose.** Removes the literal dictionaries and bare decimals; rates now appear as a `$/%` narrative the model must turn into its own data structures. **Never reaches more than 4/12.** This is the cliff.
- **L4 — Natural rules.** Re-arms the volume tier as natural bands ("under 100 … 100–999 … 1000+"). Already past the cliff: **best 3/12.**
- **L5 — Original dialect.** Also re-arms the True/False boolean — the article's original phrasing. **Best 4/12.**
- **L6 — Tables + natural tier.** Control: the L4 tier phrasing, but with the literal tables kept. **Reachable, 12/12.**
- **L7 — Tables + True/False.** Control: the L5 boolean, tables kept. **Reachable, 12/12.**
- **L8 — Tables + tier + True/False.** Control: *both* traps at once, tables kept. Still **reachable, 12/12.**

Rung	Spec dialect	Pass-rate	Best of 10	Reachable
L1	Prescriptive pseudocode	5/10	12/12	✓
L2	Declarative spec	4/10	12/12	✓
L3	Business prose	0/10	4/12	✗
L4	Natural rules (tier)	0/10	3/12	✗
L5	Original dialect (boolean)	0/10	4/12	✗
L6	Tables + natural tier	2/10	12/12	✓
L7	Tables + True/False	3/10	12/12	✓
L8	Tables + tier + True/False	2/10	12/12	✓

Table 2 · Best-of-ten (sampled, temperature 0.4) per rung. Pass-rate is how many of ten attempts hit a full 12/12; "reachable" means at least one did. Tables present (L1, L2, L6, L7, L8) ⇒ reachable; tables removed (L3, L4, L5) ⇒ not.

4 What the ladder shows — a summary

The break is the **L2 → L3 boundary**, and it is sharper than the original two-trap story. Dropping the step-by-step recipe (L1 → L2) costs nothing. Dropping the literal lookup tables (L2 → L3) is the cliff. And the failures are *not* the tier or boolean at all: at L3–L5 the model handles both correctly in prose. Inspecting *which rows pass* at the failing rungs, it is always exactly the four rows that are `non-emea` **and** `ach/wire` — the rows whose region and payment loadings are both zero, i.e. **commission = 0**. Every wrong row has a non-zero loading, and the error is precisely the missing commission: for instance an electronics/EMEA row scores a ratio of `0.8929 = 1/1.12`, exactly the effect of dropping `commission = (region + payment loading) × 1.2` from the subtotal.

The load-bearing scaffold is the literal lookup tables, not the phrasing of the famous traps. With the tables present, every dialect is reachable (L1, L2, L6, L7, L8 → 12/12); with the tables removed, none is (L3, L4, L5). The tables are necessary and nearly sufficient; the tier and boolean traps are neither necessary nor

sufficient — even both together (L8) still pass. Once the model must build its own data structures from prose, what collapses is the multi-step *commission* chain, not the two single-line constructs.

Takeaway for a spec author. You may write the ten-rule spec in plain declarative prose — but you must (a) hand over the lookup tables verbatim and (b) name every multi-step derived quantity (here, the commission) as its own explicit sub-step. Narrating a multi-step formula in one breath is the thing a 3B cannot reliably reconstruct.

Two methodology notes. First, *single greedy decoding is a knife-edge*: a one-line change to a spec's title flipped L1's deterministic output from 12/12 to 0/12, so a small model must be evaluated by sampling, not one greedy roll. Second, *the instruction wrapper can dominate the result*: an ambiguous "strip spaces from headers" made the model delete the space inside `payment type` and crash, and "never crash" induced `try/except` that swallowed the real bug — both were removed so the wrapper is precise and identical for every rung.

5 Can a repair loop rescue the failing rungs?

The repository's standard loop generates code, scores it, and feeds the failure back for another attempt. Its feedback is a **scalar total diff** ("row 3: got X, expected Y") — but the L3 failure is a *localizable structural omission*: the whole commission chain is dropped. A single wrong number cannot say *which* of ten steps is missing, so the model flails. We therefore built `--repair-rules`, a **white-box loop**: a reference computes every intermediate in the pipeline, and on each failing row the loop reports the *first* step that diverges — e.g. *"row 0: correct through `payment_loading`, but `commission` = 0.00, should be 3206.25"* — asking the model to expose those intermediates so they can be compared.

We ran it on the three failing rungs (three trajectories, up to six iterations) as an A/B on the repair temperature. The localization works — the model does fix the flagged step. At the default temperature (0.6) the score paths *oscillate* (`[4, 0, 1, 0, 0, 9]`): fixing one step regresses another, and nothing converges. **Dropping the repair temperature to 0.3** makes the edits conservative enough to stick.

Rung	Spec dialect	best-of-10 (no repair)	repair-rules @ temp 0.6	repair-rules @ temp 0.3
L3	Business prose	4/12	8/12 (0/3)	7/12 (0/3)
L4	Natural rules (tier)	3/12	4/12 (0/3)	8/12 (0/3)
L5	Original dialect (boolean)	4/12	9/12 (0/3)	12/12 (1/3, iter 2)

*Table 3 · White-box repair on the failing rungs. Cell shows best score and, in parentheses, trajectories (of three) that reached a full 12/12. At temperature 0.3 the boolean rung **L5 converges to 12/12 in a single repair iteration** (path `[4, 12]`); L4 climbs to 8/12; L3 still resists.*

Interpretation. White-box, per-step feedback is a *real* lever — enough to rescue the boolean rung and nearly the tier rung at a low temperature — but **not a guarantee** for a fully de-scaffolded spec. Rebuilding the entire commission chain from pure prose (L3) remains out of reach in this budget, and the extra burden of emitting a dozen intermediate values every turn itself destabilizes a 3B's generation (the residual zero-scores are crashes). The dependable fix for L3 is still the *spec* — put the table back, or run a spec-repair loop that rewrites the prose toward the L1/L2 dialect — rather than out-arguing the model with cleverer diffs.

6 Conclusion

Yes — a tiny AI on an 8 GB MacBook can write real, numerically-correct business logic, *provided the specification is written in its dialect*. Holding the model fixed and moving only the words, we located the boundary precisely: correctness survives plain declarative prose but collapses the moment the author stops handing over the data tables and expects the model to assemble a multi-step derived quantity from narrative. The two constructs long blamed for the failure turn out to be handled fine; the true fragility is structural composition. A white-box repair loop can push the frontier outward — rescuing the boolean rung to a perfect score at a conservative temperature — but the reliable, general lever remains the one this line of work started from: **engineer the spec, not the model.**

Model: `mlx-community/Qwen2.5-Coder-3B-Instruct-4bit` via `mlx-lm 0.29.1` (Python 3.9, Apple Silicon). Dataset: 12 rows, tolerance 0.5. Ladder metric: best-of-10 at temperature 0.4. Repair: iteration-1 greedy, subsequent iterations sampled. All specs, code, datasets, and result JSONs are in `experiments/sweet-spot/`; this report is generated from those artifacts by `docs/build_report.py`.